M17 - Data Engineering Architecture & System Design

LEARNER BOOK - BEGINNER-FIRST TEACHING RC2

Primary teacher

The lesson explains from zero and gives a concrete case before asking for proof. Videos/docs are reinforcement, not a rescue requirement.

Contents

DE17-01 - From requirements to architecture

What you will be able to do

Turn business/SLA/data/security constraints into architecture requirements before selecting products.

Why this matters

Architecture diagrams without requirements are decoration; tradeoffs only make sense against a workload.

Picture it this way

Design a bridge only after knowing traffic, span, weather, budget and safety requirements.

Start from zero

- Collect sources/formats/volumes/change rate and data growth.
- Define latency/freshness SLA and consumer query patterns/concurrency.
- Define correctness/replay/history/retention requirements.
- Classify sensitivity/residency/access needs.
- Define team skills/operations maturity/budget constraints.
- Separate hard requirements from preferences and assumptions; record unknowns.

Teacher demonstration / case

```
requirements_case.md -> requirements table:
R1 freshness <= 60 min (hard)
R2 daily volume 200 GB, 2x/year growth
R3 customer PII restricted
R4 replay last 30 days
R5 BI + ad-hoc Spark users
R6 small team, managed ops preferred
```

Read it like English

- Each architecture choice should cite one or more requirements.
- Unknown assumptions become risks/experiments, not hidden facts.

Expected check

category	example
functional	ingest API + files
SLA	60-min freshness
scale	200 GB/day
security	PII restricted
operational	small team

Common beginner traps

- Starting with "use Kafka/Spark/Kubernetes".
- Treating projected scale as current fact.
- No failure/recovery requirement.

Now you do a similar task

Read requirements_case.md and rewrite it as measurable hard/soft requirements + unknowns.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

What architecture decision becomes impossible to defend without a latency/freshness requirement?

Answer in your own words before continuing.

DE17-02 - Batch vs streaming decisions

What you will be able to do

Choose batch, micro-batch or streaming from latency/event semantics/cost/complexity and backfill needs.

Why this matters

Streaming adds operational/state complexity; batch may miss required event latency.

Picture it this way

Choose delivery frequency like transport: daily truck, hourly van or continuous conveyor based on SLA/value.

Start from zero

- Start from business latency: seconds/minutes/hours/daily.
- Consider source availability (CDC/event stream vs snapshot/file/API).
- Streaming needs offsets/checkpoints/idempotency/schema/replay/lag operations.
- Batch is easier to recompute/test and often cheaper when SLA allows.
- Hybrid is common: streaming operational feed + batch reconciliation/backfill.
- Do not choose Kafka solely because source volume is large.

Teacher demonstration / case

```
latency_cases.json
# For each case classify:
requirement -> batch/microbatch/stream -> why -> recovery/backfill
```

Read it like English

- The exercise forces a decision per SLA, not one universal architecture.

Expected check

SLA/use	likely tendency
daily finance	batch
5-min monitoring	micro-batch/stream
sub-second fraud action	streaming
historical rebuild	batch/backfill even in stream system

Common beginner traps

- Streaming for prestige.
- Batch when event response truly must be seconds.
- No reconciliation/backfill in streaming architecture.

Now you do a similar task

Classify supplied latency cases and include one hybrid solution.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

Why can a streaming architecture still require batch pipelines?

Answer in your own words before continuing.

DE17-03 - Warehouse vs lakehouse

What you will be able to do

Choose warehouse, lakehouse or combination from data types, SQL concurrency, open-format/Spark needs and governance.

Why this matters

Both can solve analytics; architecture depends on workload and operating model.

Picture it this way

Choose the storefront and warehouse system based on what goods/users need, not which label is newer.

Start from zero

- Warehouse: SQL-centric managed relational analytics/concurrency/governance.
- Lakehouse: object/lake storage + transactional table format, flexible Spark/data science/open formats.
- Combination can use lakehouse for large flexible processing and warehouse/SQL serving where needed.
- Duplication adds cost/lineage/freshness complexity; justify every copy.
- Evaluate write patterns, schema evolution, BI concurrency and tool skills.
- Avoid categorical claims that one architecture always replaces the other.

Teacher demonstration / case

```
workload_matrix.json
score needs:
- SQL BI concurrency
- Spark/ML/open file access
- semi-structured/raw retention
- cost/ops skills
- governed serving
```

Read it like English

- The matrix converts tradeoffs into explicit decision evidence.

Expected check

need	warehouse/lakehouse tendency
high SQL BI concurrency	warehouse strong
open Parquet/Delta Spark	lakehouse strong
raw multi-format	lake strong
both	hybrid with clear ownership

Common beginner traps

- Marketing-driven choice.
- Duplicating all Gold into both.
- No source-of-truth decision.

Now you do a similar task

Score supplied workload matrix and write one primary + one rejected design with reasons.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

If both warehouse and lakehouse contain Gold sales, what ownership problem must you solve?

Answer in your own words before continuing.

DE17-04 - Managed service vs build yourself

What you will be able to do

Decide managed service vs self-managed/build based on differentiation, team skills, reliability burden, lock-in and cost.

Why this matters

Building infrastructure yourself gives control but creates 24/7 operational responsibility.

Picture it this way

You can own/build a power generator or pay for utility power; choose based on what you need to control and can operate.

Start from zero

- Managed service reduces patching/scaling/HA burden but may cost more per unit and create service limits/vendor coupling.
- Self-managed gives customization/control but team owns upgrades, security, availability, backups and on-call.
- Differentiate between managed open ecosystem and proprietary feature lock-in.
- Use total cost of ownership including engineering/on-call time, not only cloud bill.
- Choose self-managed when a real requirement justifies it, not to prove technical ability.
- Exit/portability strategy matters for critical data/logic.

Teacher demonstration / case

```
responsibility_matrix.json
# Mark provider/team ownership for:
patching, scaling, backups, schema/data quality,
IAM, monitoring, disaster recovery, upgrades
```

Read it like English

- A responsibility matrix exposes hidden operational work.

Expected check

decision	question
managed	which operations transferred?
self-managed	who owns on-call/upgrades/security?
lock-in	what data/code can move?
cost	include people + failure cost

Common beginner traps

- Comparing only monthly service price.
- Assuming managed means no incidents.
- Building Kubernetes/Kafka cluster for learning into a tiny production team.

Now you do a similar task

Fill responsibility matrix for managed Kafka vs self-managed Kafka and make a recommendation for a small team.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

Which responsibilities always remain with the data team even with a fully managed storage/compute service?

Answer in your own words before continuing.

DE17-05 - Scaling and bottleneck reasoning

What you will be able to do

Find the bottleneck from evidence and scale the constrained component instead of scaling everything.

Why this matters

Systems scale until one resource/dependency limits throughput/latency. Randomly adding nodes increases cost without fixing the bottleneck.

Picture it this way

A factory output is limited by the slowest station, not the average number of workers.

Start from zero

- Measure source rate, queue/lag, task duration, CPU/memory/I/O/network, shuffle, DB query/load throughput and file counts.
- Horizontal scale adds workers/partitions; vertical adds resources to instances; some workloads cannot parallelize beyond partitions/keys.
- A downstream DB may bottleneck a highly parallel Spark ingest.
- Skew/hot keys create single-partition bottlenecks.
- Backpressure/rate limiting protects downstream systems.
- Capacity planning uses current metrics + growth headroom/SLA.

Teacher demonstration / case

```
scale_metrics.json
# Diagnose scenarios:
Kafka lag grows but consumers CPU low -> inspect sink/partitions/errors
Spark one task 10x slower -> inspect skew
DB writes saturated -> limit parallel writers / optimize target
```

Read it like English

- The correct scaling action depends on observed constraint.

Expected check

symptom	possible bottleneck
one long Spark task	skew
all executors CPU high	compute
DB connection saturation	target
Kafka hot partition lag	key/partition skew

Common beginner traps

- Adding Spark workers for a serial DB bottleneck.
- Averaging task duration and missing one tail.
- Capacity plan with no growth assumptions.

Now you do a similar task

Diagnose supplied scale metrics and propose smallest justified scaling/change experiment.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

Why can increasing consumer count fail to reduce Kafka lag?

Answer in your own words before continuing.

DE17-06 - Reliability and failure domains

What you will be able to do

Design reliability by identifying failure domains, retries/idempotency, redundancy, RPO/RTO and dependency recovery.

Why this matters

A reliable architecture expects components/zones/jobs to fail and limits the impact.

Picture it this way

Do not ask "will it fail?"; ask "what breaks when this room/network/source fails, and how do we recover?"

Start from zero

- Failure domain is a set that can fail together: process, node, zone, region, credential, shared DB, source.
- Retries handle transient failures only when operations are idempotent.
- Redundancy across independent failure domains reduces single points but adds complexity/cost.
- RPO = acceptable data-loss window; RTO = acceptable recovery time.
- Raw history/checkpoints/backups enable replay/recovery.
- Test failure scenarios; documentation without drills is weaker.

Teacher demonstration / case

```
failure_modes.json
For each failure:
- blast radius
- detection
- automatic/manual containment
- data loss risk (RPO)
- recovery target (RTO)
- replay/restore proof
```

Read it like English

- The risk table makes reliability measurable.

Expected check

term	meaning
RPO	how much data loss acceptable
RTO	how long recovery may take
failure domain	things likely to fail together
idempotency	safe retry/replay

Common beginner traps

- Multi-zone copies with same corrupt pipeline = not corruption protection.
- Retries as reliability plan for deterministic bugs.
- No tested restore.

Now you do a similar task

Complete failure_modes.json reasoning and choose top three reliability investments by impact/SLA.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

How are high availability and data correctness/recoverability different?

Answer in your own words before continuing.

DE17-07 - Cost-aware architecture

What you will be able to do

Model major cost drivers and trade performance/freshness/storage/managed-service convenience against budget.

Why this matters

Cloud/distributed data systems can scale cost quickly through compute hours, scans, data transfer, duplicated storage and operational people time.

Picture it this way

Architecture is choosing enough trucks/warehouse space for SLA without paying for an empty fleet all day.

Start from zero

- Identify fixed vs variable drivers: capacity/cluster runtime, scanned bytes, storage, egress, API calls, orchestration runs, operational labor.
- Full scans/rebuilds can dominate compute; partition/pruning/incremental reduce work when correct.
- Streaming continuous compute may cost more than hourly micro-batch if SLA allows.
- Duplicating data across systems multiplies storage/refresh/lineage.
- Managed services trade unit cost for engineering/on-call savings.
- Cost model uses ranges/sensitivity, not fake exact future bills.

Teacher demonstration / case

```
cost_drivers.json
monthly_estimate =
 compute_hours * rate
 + storage_tb_month * rate
 + scan_or_query_usage
 + egress
 + managed_service baseline
 + ops risk/people notes
```

Read it like English

- Use scenarios low/normal/high volume and note uncertain rates.
- Architecture choice should survive a plausible growth case.

Expected check

driver	control
Spark runtime	right-size + incremental + stop idle
scan bytes	partition/pruning/model
storage copies	ownership/lifecycle
egress	keep compute/data locality
people/on-call	managed service tradeoff

Common beginner traps

- One precise cost number with no assumptions.
- Ignoring engineering labor.
- Optimizing cost by violating SLA/recovery.

Now you do a similar task

Fill cost driver model for two candidate designs and run 2x volume sensitivity.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

Why is a cost estimate more useful when it shows assumptions/ranges than one exact number?

Answer in your own words before continuing.

DE17-08 - Security and governance by design

What you will be able to do

Make security/governance architecture decisions at identity/network/data/layer boundaries before implementation.

Why this matters

Retrofitting security after all data is copied broadly is difficult and risky.

Picture it this way

Design the keycards, restricted rooms, encrypted transport and retention labels into the building blueprint.

Start from zero

- Classify data/sensitivity and minimize raw PII spread.
- Use workload identities, least-privilege RBAC and separate environments.
- Encrypt in transit/at rest using platform capabilities and manage keys/secrets appropriately.
- Separate raw restricted zones from curated masked/aggregated access.
- Record lineage/owner/retention and audit access.
- Consider network/private endpoint/firewall controls where required by threat model.
- Security controls must not block recovery/operations; emergency access should be audited.

Teacher demonstration / case

```
security_case.json
# Build threat/control table:
asset -> threat -> control -> owner -> audit evidence -> residual risk
```

Read it like English

- The supplied case focuses on design reasoning, not vendor checkbox memorization.

Expected check

asset	example control
PII raw	restricted RBAC + encryption + retention
pipeline credential	secret store/workload identity
Gold BI	masked/minimized + reader role
audit	access/run logs

Common beginner traps

- "Encrypted" used as full security answer.
- All engineers admin in production.
- Copying PII into dev for realism.

Now you do a similar task

Complete security case with least privilege and data-minimization plan plus one residual risk.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

Why is encrypting a table at rest insufficient if every analyst can query all plaintext rows?

Answer in your own words before continuing.

DE17-09 - Promotion foundations: CDC, Contracts, Vault, Mesh, Kubernetes

What you will be able to do

Recognize advanced promotion concepts - CDC, data contracts, secret vaults, data mesh and Kubernetes - and know when each solves a real problem.

Why this matters

Junior engineers should understand the vocabulary/tradeoff without prematurely deploying complex platforms.

Picture it this way

These are specialized tools/organizational patterns: learn what problem each addresses before adding it to the architecture toolbox.

Start from zero

- CDC captures row-level changes from source logs/streams for incremental/low-latency replication.
- Data contracts formalize producer-consumer schema/semantic/SLA expectations with version/change process.
- Secret vault/manager centralizes protected credentials/rotation/audit.
- Data mesh is an organizational/domain ownership approach with data-as-product/platform/governance ideas, not a storage technology.
- Kubernetes orchestrates containers/services; it adds operational complexity and may be unnecessary when managed/serverless services suffice.
- Promotion skill is deciding **not** to use a technology when requirement does not justify it.

Teacher demonstration / case

```
Decision prompts:
CDC? Need low-latency changed rows from operational DB?
Contract? Multiple producer/consumer teams + breaking-change risk?
Vault? Rotating shared service secrets/central audit?
Mesh? Organizational scaling/domain ownership problem?
Kubernetes? Many containerized services needing custom orchestration/control?
```

Read it like English

- Each concept maps to a problem class, not a seniority badge.

Expected check

concept	problem
CDC	change capture/incremental
contract	producer-consumer compatibility
vault	secret management
mesh	organizational data ownership
Kubernetes	container workload orchestration/platform

Common beginner traps

- Adding Kafka/K8s because job descriptions mention them.
- Calling data mesh a product you install.
- CDC without delete/order/replay semantics.

Now you do a similar task

For five scenarios, decide whether each advanced concept is justified or overengineering.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

What is one scenario where Kubernetes would add more risk/cost than value to a data pipeline?

Answer in your own words before continuing.

DE17-10 - Architecture review and defense

What you will be able to do

Write an architecture decision record, diagram/data flow, risk/cost/security plan and defend tradeoffs against alternatives.

Why this matters

Architecture review tests your ability to explain assumptions and reject alternatives, not draw the most boxes.

Picture it this way

A design defense is presenting the bridge blueprint and showing how each major choice satisfies traffic/safety/budget constraints and what could still go wrong.

Start from zero

- Start review with requirements/assumptions, then data flow/components.
- For each major decision state chosen option, alternatives, why, consequences and revisit trigger.
- Show grain/state/replay/quality ownership in data flow, not only product names.
- Include failure modes/RPO/RTO, security boundaries and cost drivers.
- List risks/unknowns and experiments/monitoring to validate them.
- Answer reviewer challenge by changing design if evidence shows assumption wrong; defense is not stubbornness.

Teacher demonstration / case

```
architecture_decision.json / architecture.md
Decision: hourly micro-batch, not streaming
Because: 60-min SLA; source API batch; small team
Consequences: simpler ops; up to 60-min latency
Revisit when: SLA < 5 min or source exposes event stream
```

Read it like English

- A good ADR makes tradeoff and revisit condition explicit.
- The practical checker expects requirements, architecture, risk/cost/security and defense artifacts.

Expected check

review artifact	must contain
requirements	measurable + assumptions
architecture	data flow/ownership/state
ADR	alternatives/consequences
risk	failure + mitigation
cost	drivers/scenarios
defense	tradeoffs/revisit

Common beginner traps

- Diagram with no requirements.
- Saying "best practice" instead of consequence.
- Hiding unknowns.

Now you do a similar task

Complete architecture workbook/practical and give a five-minute defense: requirement, design, 3 tradeoffs, top failure, top cost, top security risk, rejected alternative.

How to prove it

- Save the actual artifact/answer/config/code.
- Save independent evidence/checks.
- State assumption/tradeoff/limitation.
- If you used simulator/offline work, label it honestly.

STOP - check your understanding

What should make you revise an architecture decision after it was originally reasonable?

Answer in your own words before continuing.